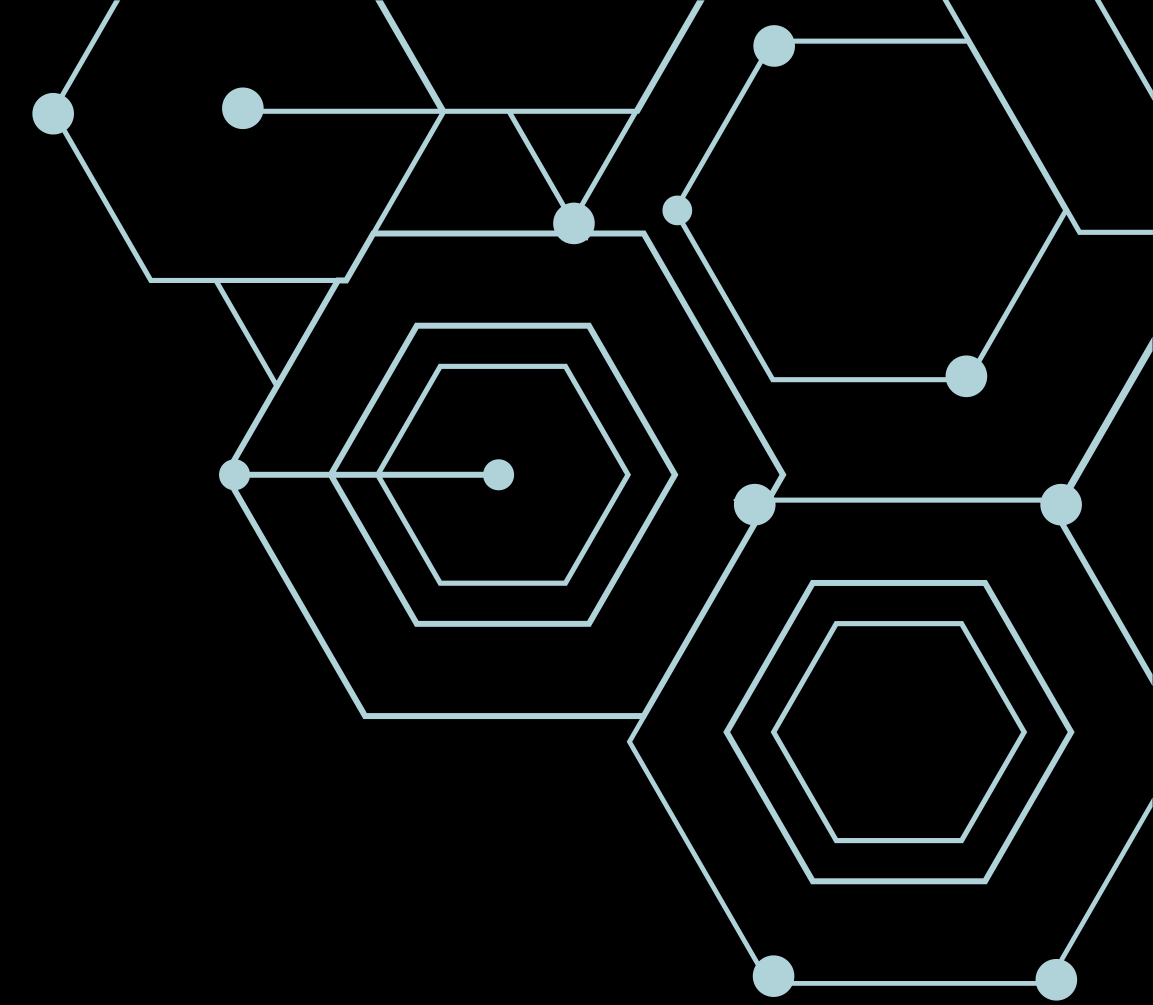




Josué Merino Calderón

**METODOLOGÍAS DE
DESARROLLO DE**

SOFTWARE

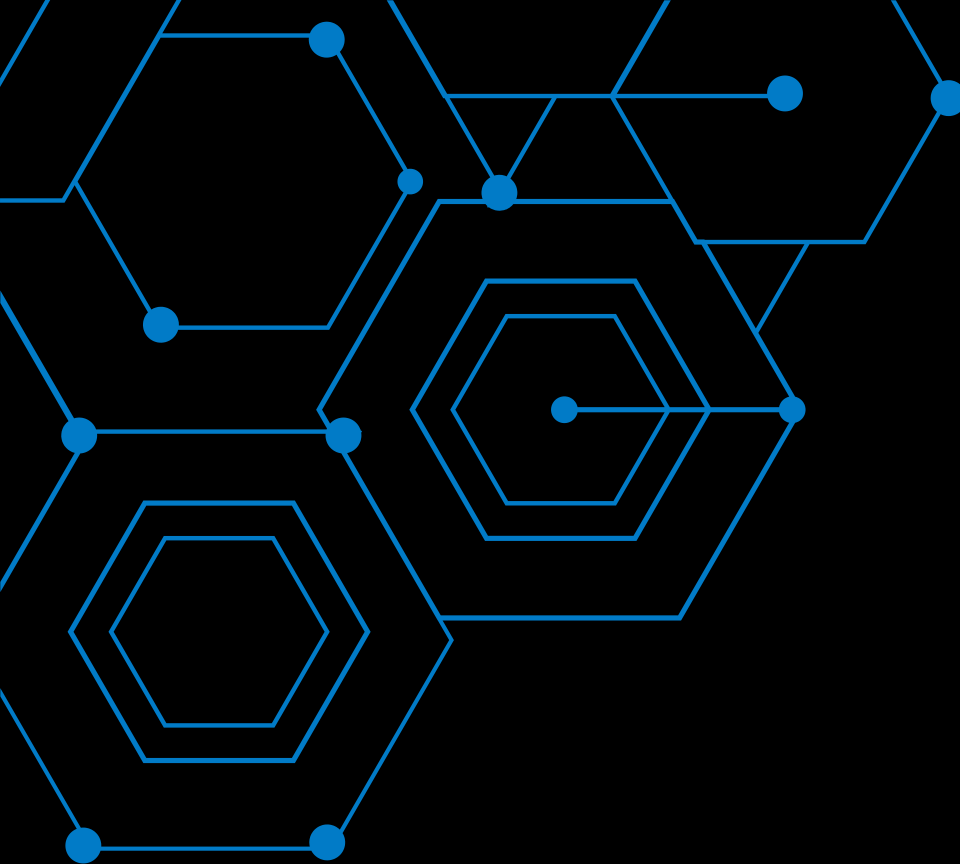
ANTECEDENTES

El desarrollo de software ha experimentado una transformación significativa a lo largo de las últimas décadas. En sus inicios, los proyectos se gestionaban mediante enfoques estructurados y lineales, donde la planificación detallada y la documentación exhaustiva eran los pilares fundamentales.

El modelo en cascada (Waterfall) fue durante muchos años la referencia principal, al establecer una secuencia rígida de fases que debían completarse de forma ordenada.

La aparición de los métodos ágiles marcó un punto de inflexión al priorizar la adaptabilidad, la comunicación constante entre los equipos y la entrega continua de valor al cliente.





OBJETIVOS

GENERAL Y ESPECÍFICOS

GENERAL

Analizar y comparar las cuatro metodologías de desarrollo de software más utilizadas en la industria actual, para identificar sus características, ventajas, desventajas y los tipos de proyectos más adecuados para su aplicación.

ESPECÍFICO 1

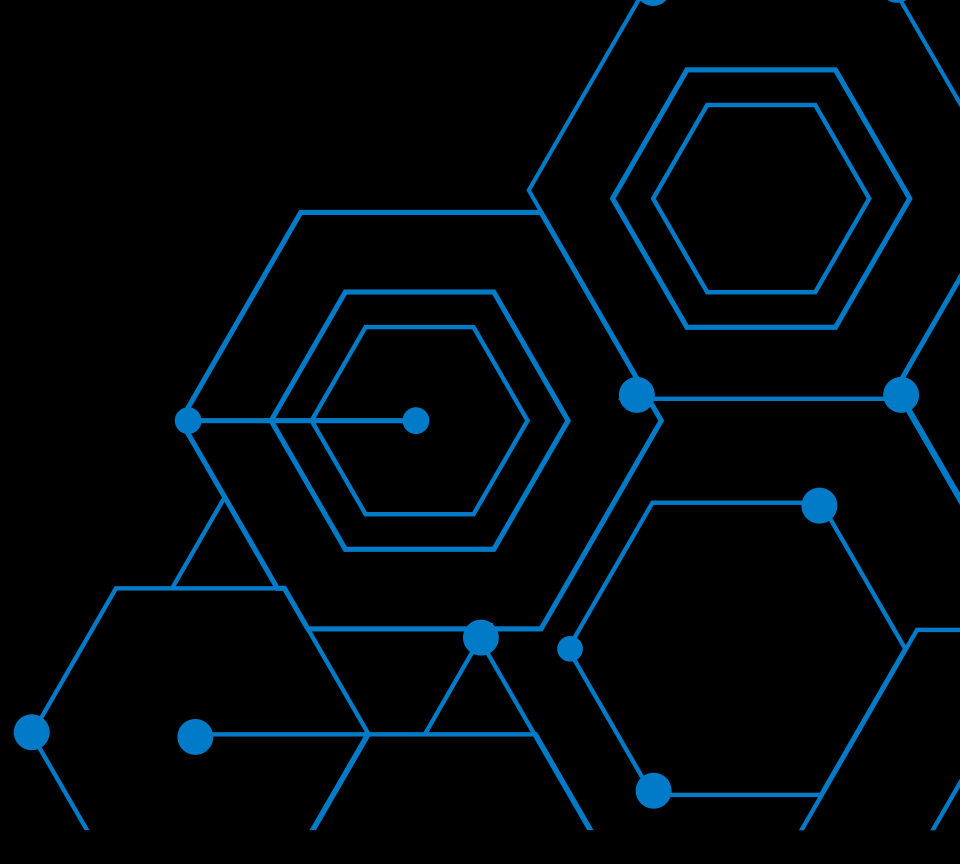
Describir detalladamente cada metodología, su ámbito de aplicación y los tipos de proyecto en los que se utiliza.

ESPECÍFICO 2

Evaluar las ventajas y desventajas de cada una de estas metodologías en el contexto del desarrollo de software.

ESPECÍFICO 3

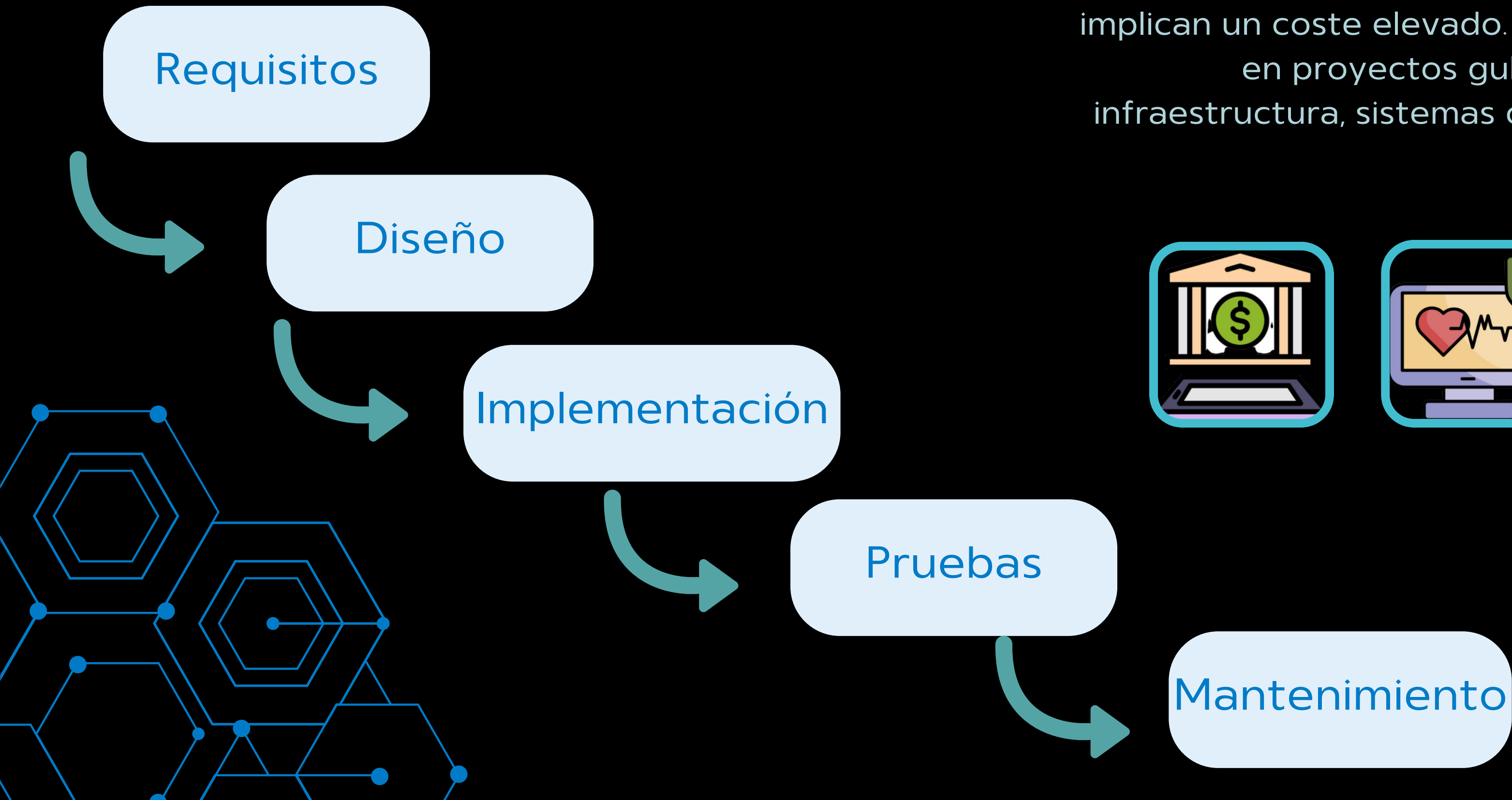
Realizar una comparación que permita facilitar en el futuro la selección de la metodología más adecuada según el tipo de proyecto, el entorno organizacional y su naturaleza.



WATERFALL

Cascada

Este enfoque se desempeña mejor en entornos donde los requisitos están claramente definidos desde el inicio, no se espera que cambien o donde los cambios implican un coste elevado. Por ello, es común en proyectos gubernamentales, de infraestructura, sistemas críticos o entornos regulados



Ventajas y Desventajas

Waterfall



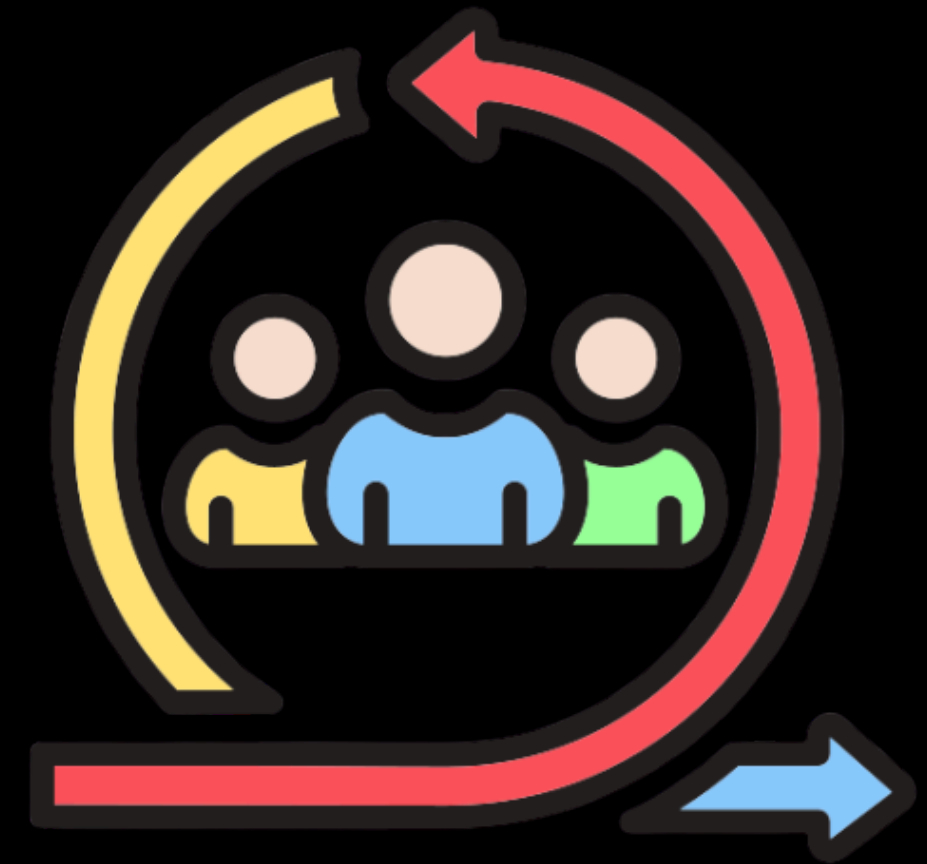
- **Claridad en la planificación y en las fases del proyecto:** cada etapa está definida y los entregables también
- **Buen nivel de documentación,** lo cual facilita rigurosidad, trazabilidad y mantenimiento.
- **Facilita la gestión** de proyectos con requisitos estables y bien comprendidos.



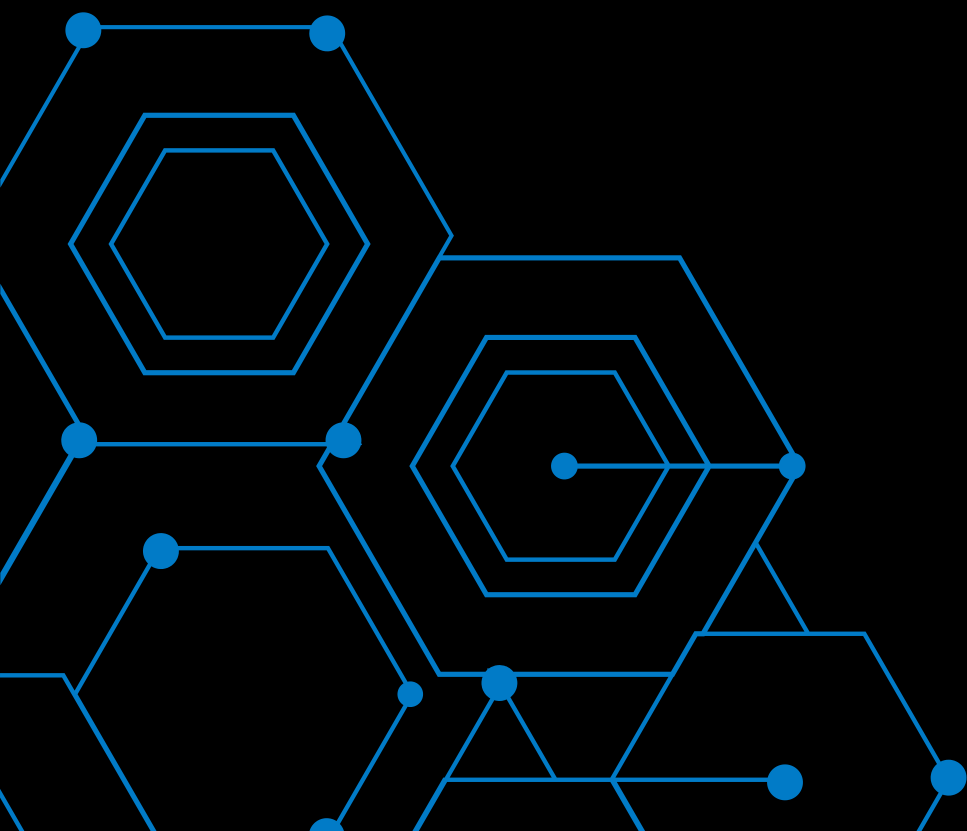
- **Muy poca flexibilidad:** si los requisitos cambian, puede generar mucho retrabajo o incluso inviabilidad del proyecto.
- **Retraso en la entrega** de valor: el usuario puede ver la solución solo al final del proceso, por lo tanto, el feedback llega tarde.
- **Riesgo de errores tardíos:** los defectos suelen detectarse en fases finales, cuando ya existe una alta inversión

SCRUM

Es especialmente útil en proyectos donde los requisitos cambian con frecuencia, como el desarrollo web, móvil o productos innovadores. Su estructura iterativa permite entregar valor de manera incremental y adaptarse rápidamente al feedback del cliente



Es un marco ágil de trabajo que organiza el desarrollo en iteraciones llamadas generalmente de una a cuatro semanas. Se basa en los pilares de transparencia, inspección y adaptación, promoviendo equipos autoorganizados que colaboran estrechamente para alcanzar objetivos comunes



Ventajas y Desventajas

Scrum



- **Alta adaptabilidad** a los cambios de requerimientos gracias a los ciclos cortos de entrega
- **Entrega de valor frecuente**, permitiendo obtener retroalimentación temprana y ajustar el rumbo.
- Mejora la **comunicación** del equipo y fomenta la participación activa de los interesados.



- Requiere un **equipo comprometido** y autoorganizado; sin ello, puede fracasar.
- Puede ser **difícil estimar presupuesto o tiempo** cuando los requisitos cambian constantemente.
- En proyectos grandes, si no se gestiona correctamente, **puede perderse** coordinación entre equipos

KANBAN

Es una metodología visual de gestión del trabajo que se centra en optimizar el flujo de tareas y reducir los cuellos de botella. Su origen proviene del sistema de producción de **Toyota** y fue adaptado posteriormente al desarrollo de software.



Se aplica con éxito en proyectos de mantenimiento, soporte y mejora continua, donde las tareas llegan de forma constante y no hay iteraciones predefinidas como en Scrum.

Ventajas y Desventajas

Kanban



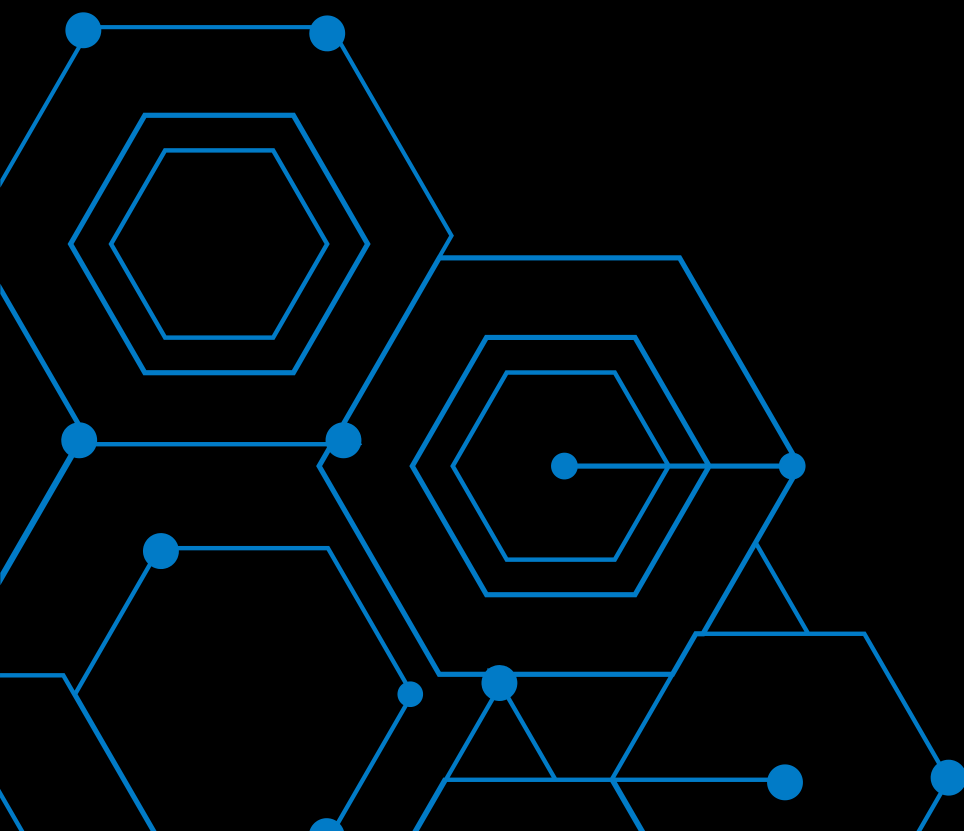
- **Visualización clara** del trabajo: el equipo puede observar el estado de cada tarea mediante tablero
- Permite **adaptarse al cambio** e incorporar nuevas tareas de forma flexible sin necesidad de reiniciar ciclos completos.
- **Control del flujo de trabajo** al limitar el trabajo en progreso (WIP), lo que reduce los tiempos de entrega.



- **Menos estructura temporal:** al no existir sprints fijos, puede perderse la disciplina o cadencia si el equipo no tiene experiencia
- **Dificultad para estimar entregas** a largo plazo en entornos con alta variabilidad.
- Requiere **seguimiento constante** de métricas (tiempo de ciclo, flujo, productividad) para lograr mejoras reales

LEAN SOFTWARE DEVELOPMENT

Deriva de los principios del pensamiento Lean aplicados originalmente a la manufactura. Su propósito principal es eliminar desperdicios, optimizar el flujo de valor, potenciar el aprendizaje y empoderar al equipo



Se utiliza principalmente en entornos donde la eficiencia y la rapidez son esenciales, como startups, proyectos de innovación o equipos que buscan mejorar procesos existentes.

Ventajas y Desventajas

Lean Software Development



- **Reducción del desperdicio:** se eliminan actividades que no generan valor, lo que mejora la eficiencia general
- Entrega **más rápida** de valor al cliente mediante la optimización del flujo continuo.
- Fortalece la **autonomía** del equipo y promueve una cultura de mejora continua



- Requiere una **cultura organizacional madura**, con equipos comprometidos con la mejora constante.
- La **flexibilidad excesiva** puede generar ambigüedad en los objetivos o entregables si no se gestionan adecuadamente.
- Puede provocar **sobrecarga de optimización** si no se equilibra con las metas reales del proyecto.

CONCLUSIONES

C1

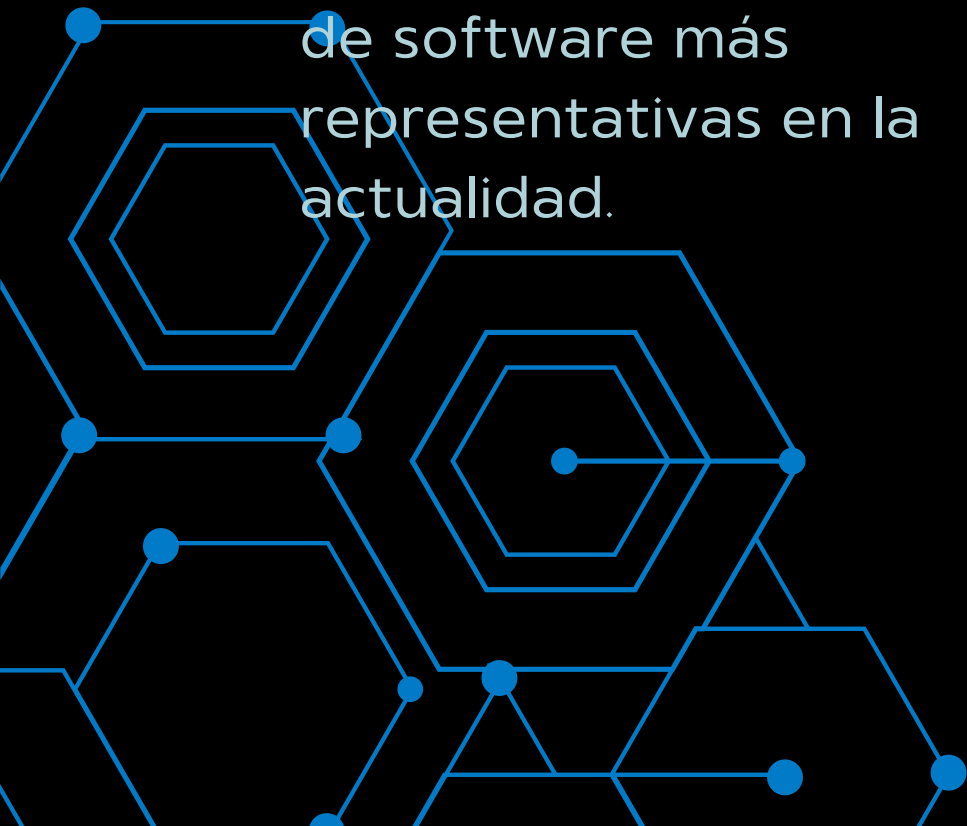
El análisis realizado permitió obtener una visión clara sobre la evolución y aplicación de las metodologías de desarrollo de software más representativas en la actualidad.

C2

Las metodologías tradicionales, como Waterfall, mantienen su vigencia en proyectos con requerimientos bien definidos y estructuras formales que demandan control y documentación precisa. En contraste, los enfoques ágiles como Scrum, Kanban y Lean se adaptan mejor a entornos dinámicos

C3

Mientras Waterfall garantiza orden y trazabilidad, Scrum fomenta la innovación continua y la retroalimentación rápida. Kanban optimiza el flujo de trabajo y la eficiencia operativa, y Lean impulsa una cultura de mejora constante centrada en la reducción de desperdicios y la entrega de valor real al cliente.





REFERENCIAS

BIBLIOGRÁFICAS



- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (8a ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Ingeniería del Software (10a ed.). Pearson Educación.
- Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.
- Poppendieck, M., & Poppendieck, T. (2003). Lean Software Development: An Agile Toolkit. Addison-Wesley.
- Scrum Alliance. (s.f.). ¿Qué es Scrum? Recuperado de <https://www.scrumalliance.org/about-scrum>
- Amazon Web Services. (s.f.). ¿Qué es Scrum? Recuperado de <https://aws.amazon.com/what-is-scrum/>
- Radigan, D. (s.f.). Kanban: How the Kanban methodology applies to software development. Atlassian. Recuperado de <https://www.atlassian.com/agile/kanban>
- Lucidchart Blog. (s.f.). Agile vs. Waterfall vs. Kanban vs. Scrum. Recuperado de <https://www.lucidchart.com/blog/agile-vs-waterfall-vs-kanban-vs-scrum>
- Lutkevich, B., & Silverthorne, V. (2021, diciembre 6). Lean software development. TechTarget. Recuperado de [Las metodologías tradicionales, como Waterfall, mantienen su vigencia en proyectos con requerimientos bien definidos y estructuras formales que demandan control y documentación precisa. En contraste, los enfoques ágiles como Scrum, Kanban y Lean se adaptan mejor a entornos dinámicos](#)

**GRACIAS POR
LA ATENCIÓN**

